

TARGET: <https://launch-ready-demo.ai> • DNS Safe Harbor Verified

Sample Security Assessment Report

This public sample shows the exact style of report Hack My Website generates for AI-built products: specific findings, affected URLs, business impact, evidence, and fix-ready guidance.

SCAN DATE
19 May 2026

TOTAL FINDINGS
10 Advisories

REPORT TYPE
Founder + Developer

AI LAUNCH SCORE

68 /100

■ HIGH RISK BEFORE LAUNCH

Security signals indicate potential vulnerabilities before production launch.

CRITICAL RISK	HIGH RISK	MEDIUM RISK	LOW / INFO
2	3	3	2
Immediate review before launch	Strong real-world misuse chance	Meaningful weakness to patch	Resilience hardening items

EXECUTIVE SECURITY SUMMARY

This sample report shows the kind of issues we regularly expect on fast-built AI websites: weak headers, exposed assets, unsafe admin surfaces, and overly trusting browser/API behavior. None of these are theoretical. A motivated attacker could use them to map the application, steal session context, or move toward sensitive customer workflows.

WHAT MATTERS MOST:

- A public admin/debug path was discoverable without meaningful friction.
- The frontend exposed build artifacts that make reverse-engineering easier.
- Browser protections were incomplete, increasing exploitability for XSS and clickjacking.

FIX FIRST PRIORITIES:

- Lock down admin/debug routes behind IP allowlists or auth.
- Disable public source maps from production hosting.
- Add a strict Content-Security-Policy (CSP) header.

[CRITICAL] Finding #1: Hardcoded connection string in backend/.env

Tool: GitHub scan + Semgrep rule • Confidence: High • OWASP: A02:2021 - Cryptographic Failures

Affected URL: GitHub: repository root -> backend/.env

WHAT THIS MEANS

A hardcoded database connection URL containing raw database credentials was found in the backend environment configuration file.

BUSINESS IMPACT

Anyone with read access to the environment file or codebase can directly log in to the database, retrieve customer records, or delete data.

TECHNICAL EVIDENCE

Semgrep pattern match: DATABASE_URL=postgresql://postgres:secret_password_123@db.supabase.co:5432/main

RECOMMENDED REMEDIATION

- Remove the database connection string from the repository.
- Store credentials in a secure environment vault or server secrets.
- Rotate all exposed database passwords immediately.

```
DATABASE_URL=os.environ.get('DATABASE_URL') # Secure env var
```

■ CLAUDE / CURSOR CODE FIX PROMPT (Copy & Paste into AI Editor)

Review backend/.env and remove hardcoded DATABASE_URL credentials. Replace with os.environ.get('DATABASE_URL') and ensure .env is gitignored.

[HIGH] Finding #2: Outdated vulnerable NPM packages detected (axios < 1.6.0)

Tool: GitHub scan + dependency-check • Confidence: High • OWASP: A06:2021 - Vulnerable Components

Affected URL: GitHub: repository root -> frontend/package.json

WHAT THIS MEANS

The package.json lists outdated package versions containing known public CVE security vulnerabilities (CVE-2021-3749).

BUSINESS IMPACT

Attackers can exploit known vulnerabilities in public library packages to perform SSRF or client-side prototype pollution.

TECHNICAL EVIDENCE

```
axios@0.21.1 contains SSRF vulnerability  
CVE-2021-3749. Recommended upgrade path:  
axios@>=1.6.0.
```

RECOMMENDED REMEDIATION

- Run `npm audit fix` to automatically patch known vulnerabilities.
- Upgrade Axios to version 1.6.0 or higher.
- Configure automated Dependabot security updates.

```
"dependencies": { - "axios": "^0.21.1" + "axios":  
                  "^1.6.0" }
```

■ CLAUDE / CURSOR CODE FIX PROMPT (Copy & Paste into AI Editor)

In frontend/package.json, upgrade axios to ^1.6.0 to remediate CVE-2021-3749 and run npm install.

[CRITICAL] Finding #3: Public admin route discoverable

Tool: Nuclei + custom check • Confidence: High • OWASP: A01:2021 - Broken Access Control

Affected URL: <https://launch-ready-demo.ai/admin>

WHAT THIS MEANS

A sensitive admin-style route was publicly reachable and exposed structure indicating privileged management surfaces exist on the public web.

BUSINESS IMPACT

If authentication controls around this path are bypassed, this becomes a direct path to user records, configuration, or administrative actions.

TECHNICAL EVIDENCE

Scanner identified a live /admin route with a recognizable login shell and response signature.

RECOMMENDED REMEDIATION

- Restrict /admin behind IP allowlists, VPN, or zero-trust identity.
- Enforce multi-factor authentication (MFA) for all admin roles.
- Rate-limit and monitor all administrative authentication attempts.

```
if not request.user.is_admin: raise
HTTPException(status_code=403, detail='Admin access
required')
```

■ CLAUDE / CURSOR CODE FIX PROMPT (Copy & Paste into AI Editor)

Secure /admin endpoint by adding strict server-side authorization checks and blocking unauthenticated public access.

[HIGH] Finding #4: Production source map exposed

Tool: Custom AI-built website check • Confidence: High • OWASP: A05:2021 - Security Misconfiguration

Affected URL: https://launch-ready-demo.ai/_next/static/chunks/app.js.map

WHAT THIS MEANS

A production JavaScript source map was publicly accessible, making the application easy to reverse-engineer and inspect.

BUSINESS IMPACT

Source maps reveal full internal TypeScript file structures, component names, API paths, and proprietary business logic.

TECHNICAL EVIDENCE

A `.map` asset returned HTTP 200 and exposed original source references from the production bundle.

RECOMMENDED REMEDIATION

- Disable production source maps in your `next.config.js` / Vite build.
- Purge already-cached `.map` URLs from your CDN edge cache.
- Re-scan after deployment to confirm source maps return 404.

```
// next.config.js module.exports = {  
  productionBrowserSourceMaps: false, };
```

■ CLAUDE / CURSOR CODE FIX PROMPT (Copy & Paste into AI Editor)

In `next.config.js`, set `productionBrowserSourceMaps: false` to prevent leaking frontend source code maps.

[HIGH] Finding #5: Missing Content-Security-Policy (CSP)

Tool: OWASP ZAP + header check • Confidence: High • OWASP: A05:2021 - Security Misconfiguration

Affected URL: <https://launch-ready-demo.ai/>

WHAT THIS MEANS

The application HTTP response did not include a Content-Security-Policy (CSP) header.

BUSINESS IMPACT

Without CSP, client-side cross-site scripting (XSS) and data exfiltration attacks are much easier to execute because the browser allows arbitrary script execution.

TECHNICAL EVIDENCE

Scanner confirmed no Content-Security-Policy header in HTTP response.

RECOMMENDED REMEDIATION

- Define a strict CSP restricting script-src, style-src, and frame-ancestors.
- Start with report-only mode if testing third-party integrations.

```
Content-Security-Policy: default-src 'self';
script-src 'self'; object-src 'none';
frame-ancestors 'none';
```

■ **CLAUDE / CURSOR CODE FIX PROMPT** (Copy & Paste into AI Editor)

Add Content-Security-Policy header to HTTP responses restricting script and object sources to 'self'.

[MEDIUM] Finding #6: Overly broad CORS policy on API route

Tool: Custom API check • Confidence: Medium • OWASP: A05:2021 - Security Misconfiguration

Affected URL: <https://launch-ready-demo.ai/api/profile>

WHAT THIS MEANS

The `/api/profile` endpoint accepted wildcard or overly permissive cross-origin requests.

BUSINESS IMPACT

Loose CORS allows malicious websites open in the victim's browser to make authenticated requests and read sensitive profile data.

TECHNICAL EVIDENCE

`Access-Control-Allow-Origin` was returned with `'*'` on an authenticated user endpoint.

RECOMMENDED REMEDIATION

- Restrict allowed origins to the exact production frontend origin.
- Never use wildcard `'*'` with credentials on sensitive endpoints.

```
allow_origins=['https://app.yourdomain.com']
```

■ **CLAUDE / CURSOR CODE FIX PROMPT** (Copy & Paste into AI Editor)

Restrict CORS middleware in backend to only allow `https://app.yourdomain.com` instead of wildcard `'*'`.